Remembering a Ball You Cannot See: Shortcuts, Memory and Attention in Occluded Pong

Giulio Sighieri
Bocconi University, Deep Learning for Computer Vision

Abstract

Agents that win Pong often read the ball's height off the opponent's paddle, which follows the ball. We hide the ball in the agent's half of the court (fog), the opponent's paddle (mask), or both, and ask whether agents then learn to remember a ball they cannot see, and how hard that is to learn. We train three PPO agents of similar size that differ in how they carry information over time: a CNN on four stacked frames, a CNN with an LSTM, and a CNN whose LSTM reads the frame through attention steered by its own memory. We test them with interventions: memory wipes, attention blocking, and edits of what the agent sees. Under fog with the opponent's paddle visible, every agent depends on that paddle: erasing it costs almost every point. With both hidden, the LSTM holds the hidden ball in its memory and acts on it: wiping the memory as the ball disappears cuts the share of balls it returns from 0.98 to 0.30. When one of the attention agent's heads tracks the ball, the agent remembers what that head lets in: blocking the head's view of the ball before the fog makes two of four models miss most balls, while removing as much attention elsewhere costs nothing. The memoryless CNN does not track the ball at all: with the ball erased from its view for the whole flight, it still returns 0.80–0.94 of the balls, having learned where they come back. The attention agent learns the fully occluded view from scratch with one seed of three, but with all three after pre-training on the clean game.

1 Introduction

Pong was among the first Atari games solved from pixels by deep reinforcement learning [12, 13], and it remains the usual first test for a new agent. Winning it, however, does not mean the agent has understood it. Delfosse et al. [6] found that agents trained on Pong track the opponent's paddle rather than the ball. The built-in opponent follows the ball closely, so its height is a good proxy for the ball's. HackAtari [5] confirmed the diagnosis: when the opponent stops moving, the agents fall apart. This is shortcut learning [7]: a cue that works on the training distribution replaces the one the task is about.

We use this to build a controlled test of memory. A *fog* hides the ball in the agent's half of the court, so the ball vanishes on its way towards the agent and reappears only at its paddle. A *mask* hides the opponent's paddle. With both, the agent cannot see the ball when it matters, nor anything that tracks it. We ask two questions. *Do agents learn to remember a ball they can no longer see, using the tools their architecture gives them?* And *how hard is that memory to learn?*

We compare three agents of similar size that differ only in how they carry information over time: a **CNN** on four stacked frames, the DQN setup; an **LSTM** agent, which sees one frame per step and keeps a recurrent memory [8]; and an **attention** agent, a simplified version of Mott et al. [14], whose memory decides where on the frame its attention looks. We train each on the four combinations of fog and mask and test them with interventions rather than correlations.

Findings.

1. **The shortcut survives the fog.** With only the ball hidden, every agent depends on the opponent's paddle: erasing or mirroring it costs almost every point, and moving it by 4–8

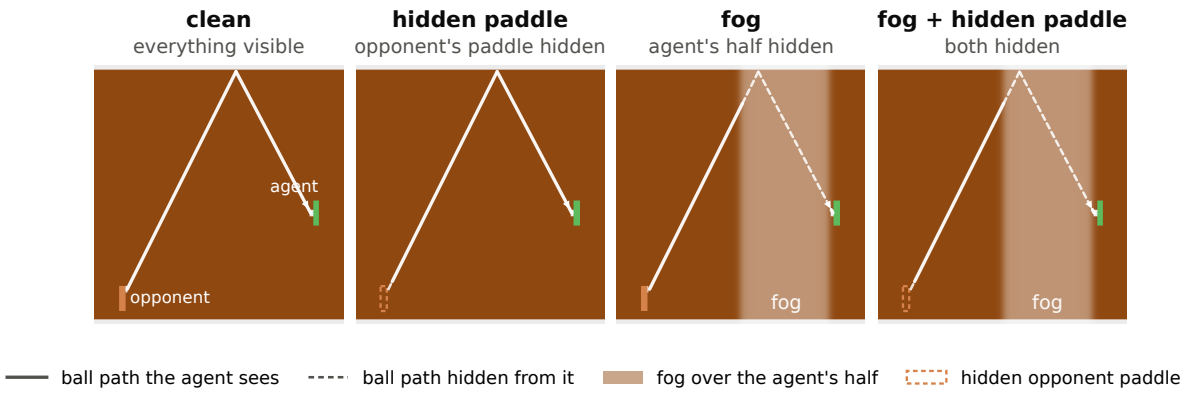


Figure 1: The four views of Pong, on the game screen. One ball flight goes from the opponent's paddle (left) to the agent's (right), bouncing off the top wall. The path is solid where the agent's view shows the ball and dashed where it hides it. The fog covers the agent's half of the court (shaded here); in the agent's own 84×84 grayscale frame it is painted in the court's colour, so the ball simply vanishes there. The mask paints over the opponent's paddle (dashed outline). The agent always sees its own paddle.

pixels most balls. It is necessary but not always sufficient: erasing the ball while it is still visible also costs most memory agents part of their returns.

2. **With the shortcut gone, the LSTM remembers the ball.** A linear probe reads the hidden ball's height from its memory (R^2 0.87–0.97, against 0.38–0.58 for an untrained LSTM), and wiping that memory as the ball disappears cuts the share of balls returned from 0.98 to 0.30.

3. **The attention agent remembers what its ball head lets in.** In models where a head tracks the ball, blocking that view before the fog removes the ball from memory; in two of four models the agent then misses most balls, while removing the same amount of attention elsewhere changes nothing.

4. **The CNN plays without seeing the ball.** On fog + hidden paddle it returns as many balls with the ball erased from its view for the whole flight (0.80–0.94, against 0.85–0.97): it has learned where the ball comes back, not where it is.

5. **The attention agent's memory is hard to learn from scratch.** It learns the fully occluded view with one seed of three, but with all three after 10M steps on the clean game and 5M on the occluded one.

Throughout, only interventions separate what an agent uses from what merely correlates with its behaviour: natural attention, decodability and early movement each turn out to be poor guides on their own.

2 Setup

Environment. We use ALE Pong [3] through Gymnasium with the standard preprocessing [11]: 84×84 grayscale frames, frame skip 4 (one agent step is four emulator frames), up to 30 no-op starts, and no sticky actions. The occlusions are applied to the observation only, so the game itself is unchanged. Figure 1 shows the four views: **clean**, **hidden paddle** (mask), **fog** (columns 42–71 of the frame), and **fog + hidden paddle**. Under fog an incoming ball is invisible for about 14 agent steps; we call the step it disappears *fog entry*. For analysis, object positions are read from the unoccluded frame, which the agent never sees.

agent	input per step	memory	parameters
CNN	4 stacked frames	none (the stack)	1.69M
LSTM	1 frame	LSTM, 128 units	2.01M
attention	1 frame	LSTM, 128 units, fed by 8 attention heads	1.98M

Table 1: The three agents. All use the Nature-CNN convolutions and the same PPO hyperparameters.

Agents. All agents share the Nature-CNN convolutions (three convolutional layers; [13]) and separate linear actor and critic heads. They differ in what they keep from one step to the next (Table 1).

- The **CNN** sees four stacked frames and maps them to 512 features. Its only memory is the stack.
- The **LSTM** agent sees one frame per step; its 512 features feed an LSTM with 128 units [9].
- The **attention** agent also sees one frame. Its convolutions produce a 7×7 map of 64 features, to which two fixed channels add each cell's x and y position. Each of eight heads scores the 49 cells with a query computed from the LSTM's previous hidden state h_{t-1}; the softmax-weighted sums of the cells' values (320 ReLU units per head) pass through a 512-unit ReLU layer into a 128-unit LSTM. This readout is the LSTM's only input, so what the agent remembers decides what it looks at next. We layer-normalise queries and keys without a learned scale; without this the attention stays uniform at initialisation and never learns where to look. The agent was widened to the LSTM agent's parameter count, so that differences between them are not a matter of size.

Training. PPO [15] follows CleanRL [10]: 16 parallel games, 256 steps per game per update, 4 epochs of 4 minibatches, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.1, entropy coefficient 0.02, and Adam with learning rate 10^{-3} annealed linearly to zero. The recurrent agents are trained on whole 256-step sequences, with the memory reset at the end of each game. Every agent is trained from scratch on every view for 15M agent steps, with three seeds. The attention agent is also trained with a *curriculum*: its clean model's 10M-step checkpoint is fine-tuned on each occluded view for 5M steps, with a fresh optimiser and the learning rate restarted and annealed over those steps, for the same 15M total. As controls, the CNN and LSTM are fine-tuned from their clean models in the same way, for 5M and for 15M steps.

Evaluation. Each frozen model plays 30 games against the built-in opponent, with environment seeds 10000–10029 and its stochastic policy. A game's return is points won minus points lost, from -21 to $+21$; a model *wins* a view if it wins most of its 30 games. The causal tests replay every ball flying towards the agent from an exact emulator snapshot, in several branches that differ only by one intervention, and compare the *share of balls returned*. A model *plays* a view if it returns at least half of the balls without intervention; a model can return 80% of the balls and still lose games, because the opponent wins long rallies. A second unmodified replay, with different action noise, agrees with the first within 0.04 everywhere.

3 Results

3.1 What each agent learns

Figure 2 shows how well each model plays its own view.

- **Every agent learns the clean, hidden-paddle and fog views**, with every seed ($+17.0$ to $+20.9$), from scratch and with the curriculum.

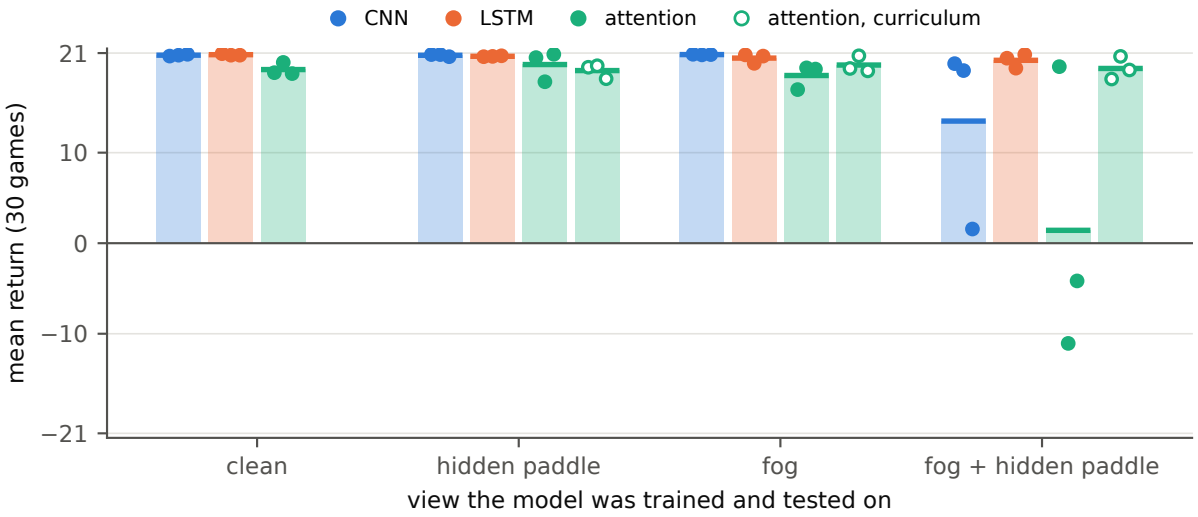


Figure 2: Mean return of each final model on the view it was trained on (bars and ticks: mean over seeds; dots: seeds; 30 games each). Hollow: the attention agent's curriculum, pre-trained on the clean view for 10M steps and fine-tuned on the target view for 5M (there is none for the clean view).

- **Fog + hidden paddle separates them.** The LSTM agent wins with every seed (+20.4, +19.3, +20.8). The memoryless CNN wins with two (+19.8, +19.1, +1.6). The attention agent wins with one (+19.5, −11.1, −4.2), but with all three after the curriculum (+18.1 to +20.6).
- **Every model is a specialist.** A fog + hidden paddle model scores about −20 on every other view, including the clean one, and a clean model about −20 on the fogged ones.

Section 3.6 returns to how these models were learned. First we ask how they play.

3.2 Under fog, every agent depends on the opponent's paddle

If the fog forced the agents to remember the ball, a fog-trained agent would not need the opponent's paddle. But it still does.

- **Erasing or mirroring the paddle** (flipping it to the opposite height) in what the agent sees makes every fog-trained model lose almost every point (−20.2 to −21.0), although the real ball still reaches its paddle.
- **A small shift is enough** (Figure 3). Moving the paddle the agent sees 8 pixels up cuts the share of balls returned from 0.90–1.00 to 0.01–0.46 in every fog model, and 4 pixels up already costs 10 of the 12 more than a third of it. Downward shifts cost less and less consistently (0.15–0.89 at 8 pixels). The clean CNN does not notice any shift (0.95 or more), so the effect is not a generic response to an edited frame.
- **Clean models differ by architecture.** With the paddle erased, the clean CNN still wins (+20.0 to +20.7); the clean LSTM and attention models depend on it to varying degrees (+18.8, −11.0, +7.5 and +15.7, −20.2, −4.6). The shortcut is not created by the fog: with the ball in full view, the memory agents already use it.
- **The fog attention models look at it.** Inside the fog, one head of every fog model turns to the opponent's paddle (0.34–0.75 of its weight, against 0.06 for uniform attention; Figure 6).

How the paddle is read remains open. The direction in which the agents move their own paddle after a shift is not consistent: of the 12 fog models, three move against the shift, four move down

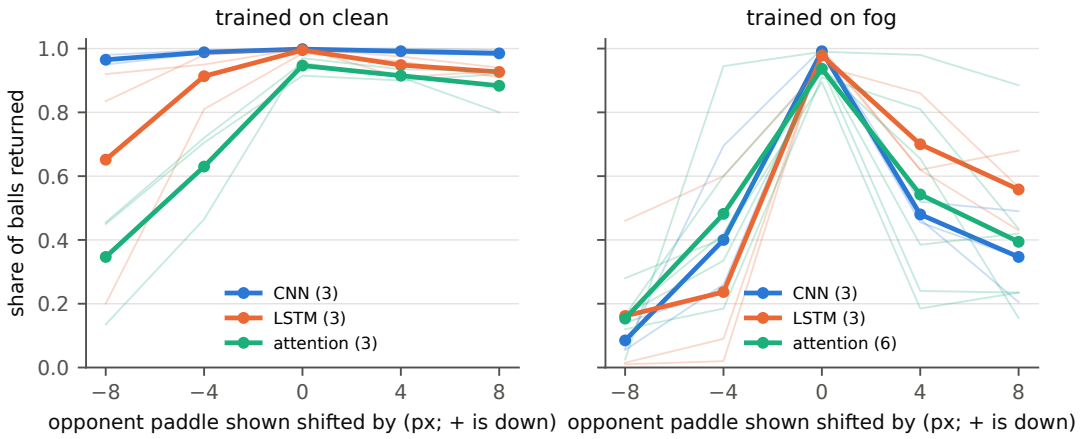


Figure 3: Share of balls returned when the opponent's paddle the model sees is moved by −8 to +8 pixels (+ is down). Only the paddle's pixels move; the ball and the game are unchanged. Thick lines: mean over models; thin lines: single models (the attention agent counts from scratch and with the curriculum). Models that do not play their view are left out. Left: models trained on the clean view. Right: models trained on fog.

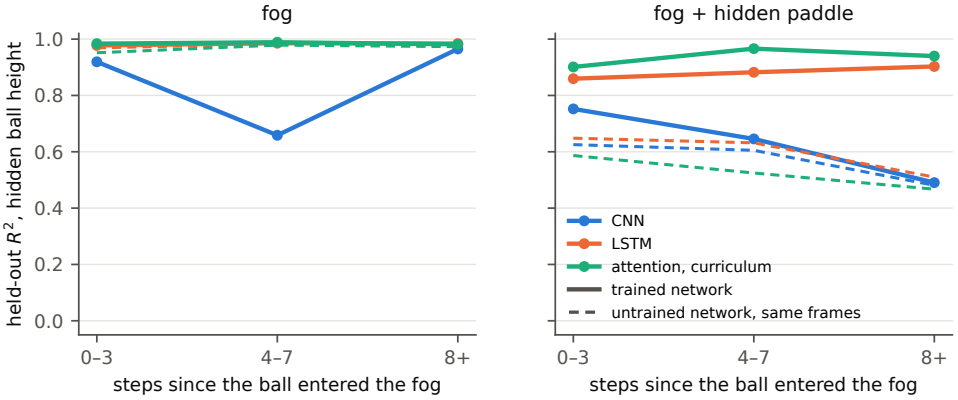


Figure 4: Held-out R^2 of a linear probe for the hidden ball's height from each model's internal state (LSTM $[h, c]$; for the CNN, its 512 features), by steps since fog entry. Solid: the trained model. Dashed: an untrained network of the same architecture fed the same frames. The attention agent is shown with the curriculum, the only variant that wins fog + hidden paddle with every seed. On fog, the dashed lines lie under the solid ones.

whichever way the paddle is shifted, and five respond in one direction only, or hardly at all, yet still miss.

The paddle is necessary, but not always sufficient. Erasing the ball while it is still visible, before the fog, also costs seven of the nine fog-trained LSTM and attention models part of their returns (from 0.88–1.00 to 0.26–0.82). With the ball erased for the whole flight, the fog CNN still returns 0.92–0.98 of the balls, from the paddle alone, while the LSTM returns 0.52–0.95 and the attention agent 0.20–0.94. Scripted players on the same games under fog put this in perspective: copying the opponent's paddle returns 0.55 of the balls, going to the ball's last seen height 0.66, and extrapolating its seen path 0.88 (0.96 without fog). The trained agents (0.90–1.00) combine both sources.

3.3 With the shortcut gone, the LSTM remembers the ball

Probes. Following Alain and Bengio [1] and Anand et al. [2], we fit ridge regressions, cross-validated over games, from each model's state to the ball's height while it is hidden. To separate

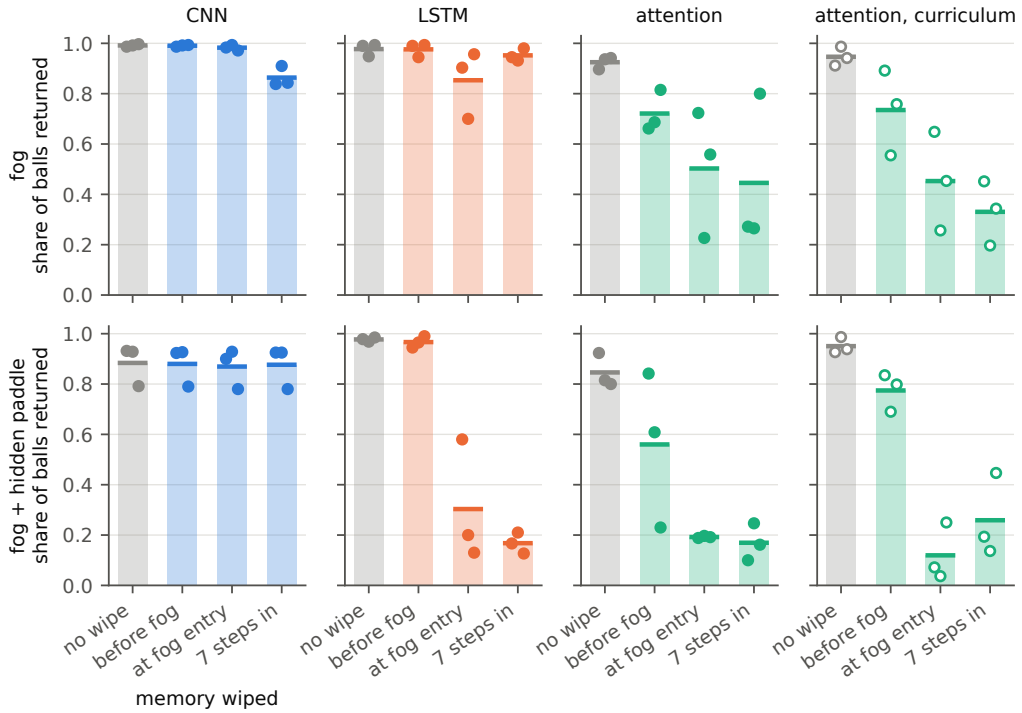


Figure 5: Share of balls returned when the memory is wiped (LSTM state set to zero; for the CNN, the frame stack filled with the current frame for four steps) at one of three moments: while the ball is still visible (as it crosses $x = 30$, a few steps before the fog); at fog entry; or seven steps after it. Rows: training view. Grey: no wipe. Bars and ticks: mean over models; dots: models; hollow: curriculum.

memory from input, every frame is also fed to an untrained network of the same architecture, with its own memory, and probed the same way (Figure 4).

- **Under fog alone, decodability is not memory.** Eight or more steps in, the hidden ball decodes almost perfectly from every model and from the untrained networks (R^2 0.95–0.99), because the visible opponent paddle tracks it.
- **On fog + hidden paddle, only trained memory holds the ball.** Eight or more steps in, the LSTM's memory gives R^2 0.87–0.97 against 0.38–0.58 for the untrained LSTM, and the curriculum attention agent's 0.92–0.97 against 0.19–0.70. The CNN's features decode the ball no better than an untrained CNN's (0.40–0.57 against 0.39–0.55).

Memory wipes. A probe shows that information is present, not that it is used. We therefore wipe the memory and replay each ball (Figure 5).

- **The fog + hidden paddle LSTM depends on its memory.** A wipe at fog entry drops the share of balls returned from 0.97–0.98 to 0.58, 0.20 and 0.13; seven steps in, to 0.13–0.21. A wipe before the fog costs nothing (0.94–0.99), because the ball is seen again.
- **The fog-only LSTM barely needs memory** (0.70–0.96 after a wipe at fog entry): with the opponent's paddle in view it rebuilds its estimate.
- **The CNN's frame stack carries nothing through the fog.** No wipe changes its share on fog + hidden paddle (0.78–0.93 in every branch).

Timescale. The memory is tuned to the occlusion. Recomputed on their own games, the forget gates of the fog + hidden paddle LSTMs give their ten slowest units half-lives of 18.5, 16.5 and

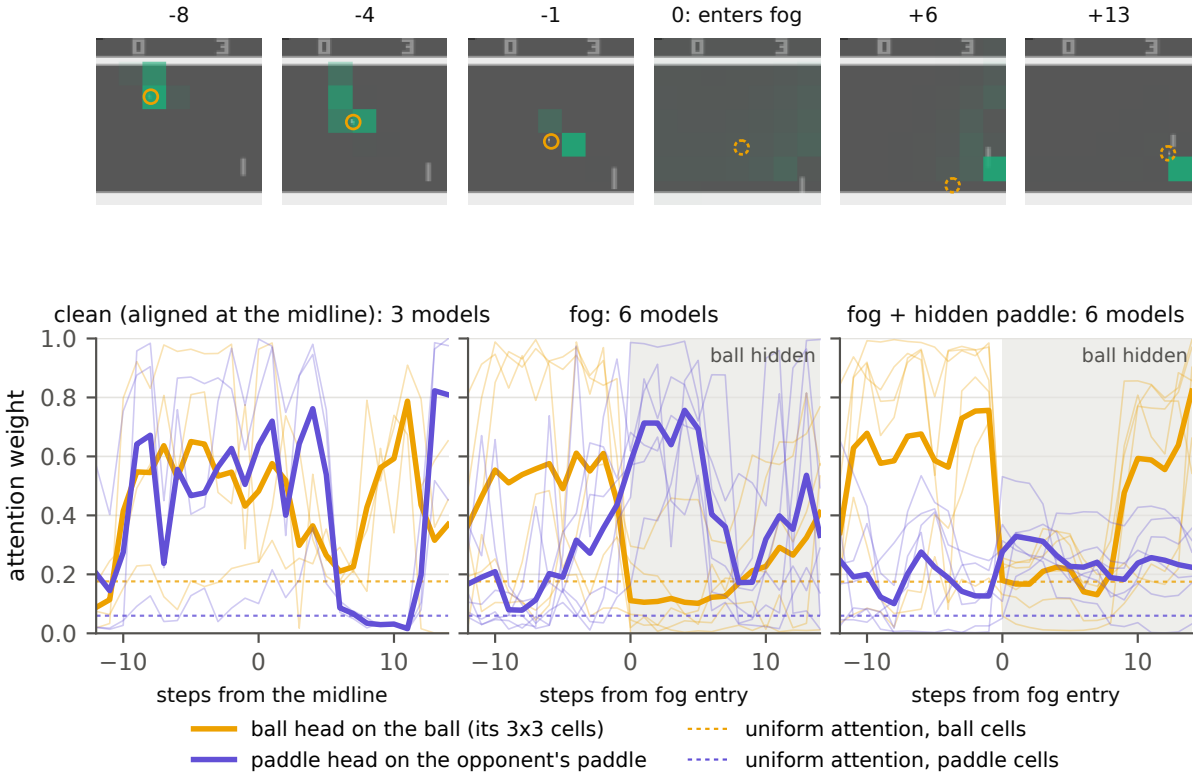


Figure 6: Where the attention agent looks while a ball comes towards it. Top: one returned flight of a curriculum model on fog + hidden paddle, on the frames the agent sees, with the weights of its ball head (green; circle: the ball, dotted while hidden). Bottom: for every model that plays its view, the ball head's weight on the ball's 3×3 cells and the paddle head's weight on the opponent's paddle, aligned on fog entry (clean: on the ball crossing the midline). Each model's heads are chosen on half of its returned flights and plotted on the other half, so choosing the best of eight heads does not flatter them. Dashed: uniform attention.

8.3 steps (medians per model), about the 14 steps the ball stays hidden. Clean LSTMs instead carry units that hardly forget.

3.4 The attention agent remembers what its ball head lets in

Wipes. The attention agent depends on its memory at least as much as the LSTM agent. On fog + hidden paddle, a wipe at fog entry leaves 0.04–0.25 of the balls returned; even on fog, with the opponent's paddle in view, it recovers less (0.23–0.72). Unlike the LSTM agent, it is also hurt by a wipe before the fog (0.23–0.89), because its memory sets its attention: a wipe both erases what it remembers and disturbs where it looks.

Where it looks. Each of its eight heads is free to look anywhere; Figure 6 follows, in each model, the head that looks most at the ball and the one that looks most at the opponent's paddle. On fog + hidden paddle, in three of the four winning models a head follows the ball cell by cell while it is visible (0.87–0.97 of its weight on the ball's cells in the eight steps before the fog, against 0.18 for uniform attention) and lets it go when it vanishes (0.02–0.26). On fog, in every model a head turns to the opponent's paddle inside the fog (Section 3.2). Natural attention is a weak guide on its own, however: per flight, attention to the ball before the fog does not predict the memory's error inside it consistently across models (Spearman -0.11 to $+0.42$).

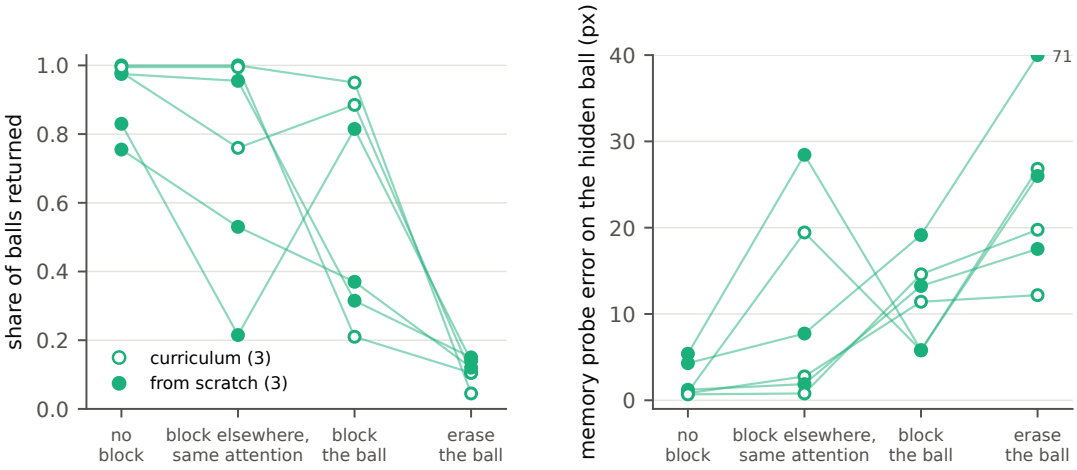


Figure 7: The attention agent on fog + hidden paddle. Each incoming ball is replayed while, as long as the ball is visible before the fog, the agent may not attend to the 3×3 cells around it (*block the ball*). The control blocks the cells with the most attention elsewhere until they hold as much attention as the ball's cells (*block elsewhere*), and *erase the ball* removes the ball from the screen. Left: share of balls returned. Right: mean absolute error of the memory probe on the hidden ball's height (one value of 71 px drawn at the edge). Lines: models; hollow: curriculum. The two from-scratch models that do not win still return 0.76 and 0.83 of the balls unblocked.

Blocking attention. The readout is the only path from the frame to the attention agent's memory, so we can ask directly whether it remembers only what it attends to. While the ball is visible before the fog, we remove the cells around it from every head's softmax; neither paddle's cells are ever blocked. As a control, a second branch removes the same attention mass from the cells the heads weight most outside the ball, and a third erases the ball from the screen (Figure 7).

- **Blocking the ball removes it from memory.** In the three winning models whose ball head tracks it, the probe's error on the hidden ball rises from about 1 px to 11–15 px. In two of them the agent then misses most balls (from 0.97 and 0.99 to 0.32 and 0.21).
- **The same attention removed elsewhere changes nothing** in those two models (0.95 and 0.99 of balls returned; probe error 1.9 and 0.8 px), so the effect is specific to the ball, not to disturbing the softmax.
- **Blocking adds nothing to erasure.** With the ball erased, also blocking its cells changes the share of balls returned by at most 0.05 in every winning model: the block acts only through the ball.
- **Not every model gates the ball through one head.** In the third tracking model the memory loses the ball (11 px) but the agent still returns 0.95 of the balls. In the model whose heads barely track the ball, blocking it costs little (0.89), while blocking its most-attended cells elsewhere costs more (0.76): it reads the ball through other cells, whose receptive fields overlap it.

3.5 The CNN does not need to see the ball

The CNN wins fog + hidden paddle with two seeds, yet no wipe of its frame stack changes anything (Figure 5), and a probe reads no more of the hidden ball from its features than from an untrained CNN's (Section 3.3). Its play suggests early commitment: four steps into the fog, its paddle is already 6–9 px from where the ball will arrive (LSTM 11–43 px, attention 12–17 px), and it makes only 4–19% of its movement in the last four steps. Two interventions show that it needs neither early movement nor the ball.

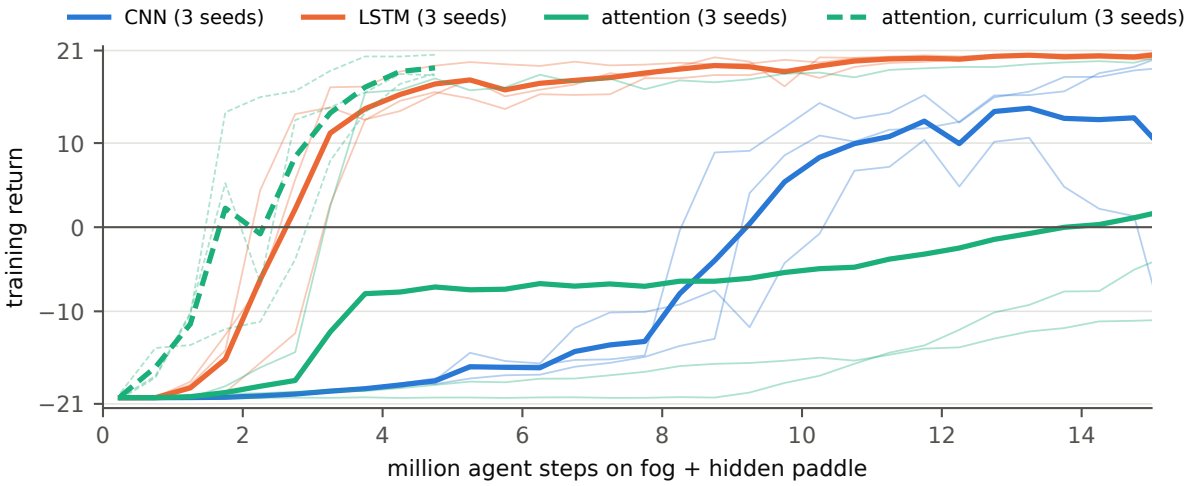


Figure 8: Training return on fog + hidden paddle against steps played on that view (mean of the training games in each 0.5M-step bin). Thin: seeds; thick: mean. Dashed: the attention agent's curriculum, whose 5M steps on this view follow 10M on the clean one.

- **Holding its paddle still** from the moment the ball leaves the opponent until four steps after fog entry, when its frame stack no longer shows the ball, barely changes the share of balls it returns (from 0.85–0.97 to 0.77–0.92).
- **Erasing the ball** from every frame it sees, for the whole flight, barely changes it either (0.80–0.94). The same erasure leaves the clean CNN 0.12 of the balls, and the fog + hidden paddle LSTM and attention agent 0.06–0.14.

The CNN does not carry the ball through the fog: it does not need to see this ball at all. Its rallies are regular, with arrival heights that vary by only 6–7 px in the winning seeds, and it has learned where the ball will come back. Regular rallies alone do not explain it: the LSTM's are as regular (3–5 px), yet the LSTM needs the ball. We have not tested how the regularity arises; the agent's own returns, which set up the opponent's next shot, are the likely source. The fog CNN goes further in the same direction: with the ball erased for the whole flight it still returns 0.92–0.98 of the balls, from the opponent's paddle alone.

3.6 Learning to remember

Figure 8 shows how the fog + hidden paddle models were learned.

- **The LSTM agent learns quickly and reliably**: every seed's training return turns positive within 2.5–3.5M steps.
- **The attention agent learns as fast, or not at all.** Its one winning seed turns positive at 3.5M steps, as fast as the LSTM. The other two stay near −20 for 4M and 10M steps and are still losing when training stops at 15M (−11 and −3 in training).
- **The curriculum makes it reliable.** Fine-tuned from its clean model, every seed turns positive within 2–3.5M steps of the 5M.
- **The CNN needs much longer** (9–11M steps), consistent with a strategy that only pays once rallies have become regular (Section 3.5).

The same 5M-step fine-tuning is not enough for the CNN and LSTM: their fine-tunes are still losing at 5M (−15.5 and −8.9 in evaluation). This is not negative transfer. Fine-tuned for the full 15M with the scratch schedule, every CNN and LSTM fine-tune wins (+13.3 to +20.9), and

two LSTM fine-tunes learn faster than their scratch runs (+9.1 and +13.8 after 2M steps, against −8.7 and −8.2). Clean pre-training is a good start for every agent; for the attention agent it is the difference between failing and winning. One explanation, which we have not tested, is a chicken-and-egg problem: the attention agent's memory holds only what its attention lets in, and its attention is steered by its memory. A clean model already has heads that track the ball, and memory can build on them.

4 Related work

Deep Q-networks learned Atari games, Pong included, from pixels with four stacked frames [12, 13]. Hausknecht and Stone [8] replaced the stack with an LSTM and showed that recurrence helps when frames are randomly blanked (flickering Pong). Our occlusion is structured instead: one region, always hidden, so that memory, a shortcut, or a strategy that needs no tracking are all possible solutions. Attention-augmented agents [14] let an LSTM query a spatial feature map, which makes where the agent looks visible; we use a simplified version and intervene on that attention rather than only inspecting it. Linear probes for Atari state follow AtariARI [2]. Shortcut learning [7] was diagnosed for Pong by SCoBots [6] and HackAtari [5]; we show that the shortcut persists under fog. Clean-then-hard training is a simple curriculum [4].

5 Limitations

- **Scale.** Three seeds per condition and a single game. The learning result rests on 1 of 3 seeds against 3 of 3 (Fisher's exact test, $p = 0.4$), and several mechanistic findings on two to four models.
- **Interventions leave the training distribution.** An erased or mirrored paddle, a blocked attention cell and a wiped memory are all states the agents never saw in training. We pair each with a control (the clean CNN under the paddle edits, attention removed elsewhere, a wipe before the fog), but a control cannot rule out every non-specific effect. The probes are fit on natural states, so their errors under an intervention are only indicative.
- **Evaluation** uses the stochastic policy against the built-in opponent only.
- **Training history.** Two runs were resumed after interruptions: the attention agent's fog + hidden paddle seed 2 after a non-finite loss at 1.34M steps (from its 1.0M checkpoint, with a fresh optimiser), and the 15M-step fine-tuning controls after power cuts. Both are recorded with the checkpoints.

6 Conclusion

Agents learn to remember the ball only when nothing else will do. With the opponent's paddle in view, even a fog over their own half leaves every agent depending on the shortcut. Without it, the LSTM agent carries the hidden ball in its memory and acts on it. The attention agent does too, and when one of its heads tracks the ball, what it remembers is what that head let in. The memoryless CNN wins without tracking the ball at all, by learning where it comes back. The attention agent, whose memory and attention depend on each other, needed the easier game first to learn to use them.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

[2] Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R. Devon Hjelm. Unsupervised state representation learning in Atari. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. doi: 10.1613/jair.3912.

[4] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009. doi: 10.1145/1553374.1553380.

[5] Quentin Delfosse, Jannis Blüml, Bjarne Gregori, and Kristian Kersting. HackAtari: Atari learning environments for robust and continual reinforcement learning. *arXiv preprint arXiv:2406.03997*, 2024.

[6] Quentin Delfosse, Sebastian Sztwiertnia, Mark Rothermel, Wolfgang Stammer, and Kristian Kersting. Interpretable concept bottlenecks to align reinforcement learning agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.

[8] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[10] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G. M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.

[11] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018. doi: 10.1613/jair.5699.

[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[14] Alexander Mott, Daniel Zoran, Mike Chrzanowski, Daan Wierstra, and Danilo J. Rezende. Towards interpretable reinforcement learning using attention augmented agents. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.